CS 449 / CS 549: Human-Computer Interaction

Human-Centred Design, Evaluation, and Interactive Systems

Final Project Handbook - Fall 2026-27

Instructor: Dr. Polat Göktaş
Email: polat.goktas@sabanciuniv.edu
Office: FENS G052

This handbook defines the final-project process, project items, assessment criteria, and topic options for CS 449 / CS 549. Every project must address a concrete HCI problem, build a working interactive prototype, include a meaningful hardware-mediated interaction, and show how evaluation changed the design.

Project support. Following topic allocation in Weeks 3, each project team will be assigned a Project TA. For questions related to your project topic, scope, prototype, evaluation plan, milestones, or deliverables, please contact your assigned Project TA first. Matters requiring an instructor decision should then be raised with the Instructor through the appropriate course channel.

Contents

1 Assessment and Milestones **2**
1.1 Final Project Assessment Components . . . 2
1.2 Milestones . . . 2

2 Group Eligibility and Topic Allocation **2**

3 Project Scope and Required Items **3**
3.1 Hardware-Mediated Interaction . . . 3
3.2 Permitted Hardware Patterns and Processes . . . 4
3.3 Required Hardware Process Description . . . 4
3.4 Expected Project Items . . . 5

4 Project Topic Pool **5**
4.1 What a Strong Project Contribution Looks Like . . . 6
4.2 Human-AI Interaction and Language Systems . . . 6
4.3 Embodied and Accessible Systems . . . 8
4.4 Trust, Safety, and Socially Situated AI . . . 11

5 Progress Presentation Requirements **14**
5.1 Progress Presentation I (Week 7) - 7% . . . 14
5.2 Progress Presentation II (Week 10) - 8% . . . 14

6 Final Prototype, Demo Day, and Final Presentation **15**
6.1 Final Prototype Submission (Week 13) 15
6.2 Final Presentation and Live Demonstration (Week 14) - 10% 15
6.3 Interactive Demo Day Format 16

7 Final Report (Week 14) - 15% **16**

8 Non-Negotiable Completion Requirements **17**

9 Ethics, Data, and Responsible Project Practice **17**
9.1 Ethics Statement and Committee Process 17

1 Assessment and Milestones

1.1 Final Project Assessment Components

The final project contributes **40%** of the course grade.

- **Progress Presentation I - 7%**, assessed by Teaching Assistants.
- **Progress Presentation II - 8%**, assessed by Teaching Assistants.
- **Final Presentation and Live Demonstration - 10%**, assessed by the Instructor.
- **Final Report - 15%**, assessed by the Instructor.

1.2 Milestones

1. **Week 2:** Project topics and rules published through SUCourse.
2. **Week 3:** Project Team Leader submits the Google Form with every member's name and student number, plus five ranked topic preferences. This is the official team-registration request.
3. **Week 7:** Progress Presentation I, prepared and presented by all members.
4. **Week 10:** Progress Presentation II, prepared and presented by all members.
5. **Week 13:** Final prototype submission by the Project Team Leader.
6. **Week 14:** Final presentation, live demonstration, and Demo Day.
7. **Week 14:** Final report submission.

2 Group Eligibility and Topic Allocation

Teams consist of **four or five students**. CS 449 and CS 549 students may form mixed teams. Each team appoints one **Project Team Leader**; this is a coordination role and does not reduce any member's responsibility. Every member must contribute visibly and speak in both progress presentations and the final presentation.

The Project Team Leader submits the Week 3-Google Form. Topic allocation follows the time stamp of complete, eligible submissions. A team receives the first available option from its five

ranked preferences. Each topic is allocated to one team only unless the instructor announces otherwise. A confirmed topic may be changed only with instructor approval and a revised scope, hardware plan, and evaluation plan.

3 Project Scope and Required Items

Each project must include: a specific HCI problem and target-user context; a focused project question or testable hypothesis; user and design requirements; a working interactive prototype; one meaningful hardware-mediated interaction; an evaluation plan; evaluation evidence; and at least one documented design iteration.

3.1 Hardware-Mediated Interaction

Suitable components include a Meta Quest or comparable standalone VR headset, smartphone camera, microphone, vibration motor, accelerometer, GPS, screen reader, wearable device, webcam, RFID/NFC card, Arduino, ESP32, Raspberry Pi, LED, button, sensor, tangible card interface, or physical mock-up. The component must change or enable the interaction. Decorative hardware does not meet the requirement.

Metaverse and XR option. Projects may use immersive, augmented, or mixed-reality interaction where it is central to the HCI problem. A suitable XR project must define the user task in the virtual or augmented environment, the interaction modality (for example gaze, controller, hand, voice, body movement, or haptic feedback), the onboarding and comfort procedure, and a non-immersive fallback for participants who cannot or do not wish to use a headset. XR should not be included only as a visual setting; the project must evaluate what the immersive or embodied interaction changes for the user.

Sabancı University Collaboration Space can support these projects through reservable VR stations and workstations, together with equipment listed in its current inventory such as a VR Headset, Eye Tracking VR Kit, Haptic Gloves, Microsoft HoloLens, Xbox Kinect Depth Camera, Depth Camera, 360-degree camera, PC workstations, and development boards. Most XR equipment is marked for on-premise use, so teams must check availability and reserve the required station or equipment in advance. Teams using a Meta Quest should confirm whether they will use their own device or a comparable available headset before finalising the project scope.

Check the Sabancı University Collaboration Space here: https://cospace.sabanciuniv.edu/en

Hardware-to-interface integration. The hardware component must be connected to the prototype through a documented software pathway. Teams must state: (i) the input or output produced by the component, (ii) the controller or device that reads or drives it, (iii) the communication method, (iv) the software runtime, and (v) the visible or audible change in the user interface. For example, an ESP32 button can send a Wi-Fi event to a JavaScript or TypeScript web interface through HTTP or WebSocket; a Python service can process sensor values and publish a state change to a browser interface; a mobile application can access camera, microphone, GPS, accelerometer, vibration, and accessibility services through its platform APIs; and an XR prototype can use Unity with C# to connect controller, hand-tracking, gaze, spatial, or haptic events to the virtual interface.

The final prototype must make the connection observable. A button, tag, sensor, gesture, voice command, or tracked movement must trigger a meaningful interface state, feedback, recommendation, safety action, or system response. Conversely, the interface may drive a physical output such as an LED, vibration, display, sound, or haptic response. The report must include an architecture figure showing this path and identify the software language and libraries used. A simulated signal is acceptable only as a backup mode; the final demonstration must include the real component working at least once.

3.2 Permitted Hardware Patterns and Processes

Teams may use their own equipment or equipment available through the Collaboration Space, subject to availability and reservation rules. The list below shows suitable implementation patterns. Teams are not limited to these patterns, but a different pattern must meet the same interaction and integration standard.

- **ESP32 or Arduino interaction controller.** Use one or more buttons, a rotary encoder, capacitive touch pad, RFID/NFC reader, distance sensor, light sensor, LED, buzzer, or vibration motor. The physical action is read by Arduino C/C++ code; the controller sends a small event such as `approve`, `token_id`, `noise_high`, or `dial_value` through serial USB, Wi-Fi HTTP, WebSocket, or MQTT. A JavaScript/TypeScript or Python service receives the event and changes the web or mobile interface. The interface may return a command to the controller, for example to illuminate an LED, play a tone, or vibrate.
- **Raspberry Pi interactive station.** Use a Raspberry Pi when the project needs a dedicated display, camera, local processing, kiosk, tabletop station, or standalone physical demonstration. Connect a screen, USB webcam, RFID reader, GPIO button, LED, or sensor. Python, Node.js, or a browser-based kiosk can process the device event and render the interface locally. This is suitable for queue displays, tangible learning stations, shared-device consent, and public interactive installations.
- **Smartphone or wearable sensing.** Use camera, microphone, GPS, accelerometer, gyroscope, vibration motor, screen reader, push notification, or smartwatch sensor. A native Android/iOS app, Flutter app, React Native app, or mobile web prototype should request permissions clearly, process only the minimum data required, and convert the sensor event into visible, spoken, or haptic interface feedback. This is suitable for navigation, speech support, accessibility, reminders, provenance capture, and mobile verification tasks.
- **NFC, RFID, QR, and tangible objects.** Use NFC cards, RFID tags, QR-coded cards, printed tokens, or physical dials when selection, permission, sequencing, or comparison benefits from tangible handling. The reader or camera converts the object into a unique identifier; the interface then shows the selected state, associated content, or consequence. The physical object must represent a meaningful user choice, not merely replace a mouse click.
- **Camera, microphone, and computer-vision interaction.** Use a phone camera, webcam, depth camera, or Kinect for OCR, object capture, gesture, posture, environmental observation, or accessibility reporting; use a microphone for speech, sound-level features, or voice commands. Teams must state whether raw audio or video is retained, where processing occurs, and how bystanders are protected. Python with OpenCV, MediaPipe, Whisper-like speech recognition, or a browser camera API are suitable starting points when used responsibly.
- **XR, spatial, and haptic interaction.** Use Meta Quest or an available VR/AR setup for controller, hand, gaze, body movement, spatial audio, or haptic interaction. Unity and C# are the recommended software route; the prototype should map each physical action to a virtual object, system state, instruction, or feedback response. Include a seated or non-immersive fallback and document comfort, motion, accessibility, and equipment-cleaning procedures.

3.3 Required Hardware Process Description

For the Progress Presentation I, each team must submit a hardware plan/progress using the following format:

1. **Hardware set:** identify the exact device and parts, for example *ESP32 + two push buttons + RGB LED*, *Android phone microphone + vibration*, or *Meta Quest hand tracking*.

2. **User action:** state what the user physically does, for example taps a consent token, turns a dial, speaks a question, scans a QR card, presses an override button, or selects a virtual object.
3. **Signal and transport:** state the event produced and how it reaches the prototype, for example NFC identifier via phone API, serial event from Arduino, WebSocket message from ESP32, or hand-tracking event in Unity.
4. **Software and interface response:** name the software language or platform and state what immediately changes in the interface, for example an explanation layer opens, an agent action pauses, an accessible prompt is announced, a recommendation is recomputed, or a physical LED confirms the setting.
5. **Fallback and test:** give a backup interaction for equipment failure and one concrete test that proves the real component changes the interaction.

The hardware, software, and interaction process stated under every topic below is an appropriate starting configuration. Teams may refine the exact hardware set after allocation, but they may not replace it with decorative hardware or remove the physical interaction without approval.

3.4 Expected Project Items

- **Problem and users:** problem statement, target users, stakeholders, and context of use.
- **Project question and expected outcomes:** focused question or hypothesis and a comparison or baseline where appropriate.
- **Requirements and design rationale:** evidence-based requirements and decisions addressing usability, accessibility, inclusion, privacy, and safety.
- **Prototype and hardware component:** a testable prototype supporting a realistic task.
- **Evaluation and iteration:** task protocol, feedback or results, analysis, and at least one revision.
- **Presentations and final report:** complete documentation of the project process and individual contributions.

Each team must complete either a user evaluation with at least **20-30 participants** and a structured expert evaluation with at least **3-5 evaluators**. Use at least two measures, such as task success, time, error rate, SUS, NASA-TLX, trust calibration, accessibility issues, or qualitative themes; check the course materials.

4 Project Topic Pool

The following are defined project directions, not titles to be copied without further work. Each team must turn its allocated direction into a precise problem statement, target-user definition, interaction flow, prototype, and evaluation protocol. These topics have been selected because a strong execution can address current Human-AI Interaction, interactive systems, accessibility, language technology, multimodality, and responsible-AI discussions at venues such as ACL and IJCAI. The expected standard is a clear problem, a defensible interaction idea, a working technical system, and empirical evidence that supports a bounded claim.

4.1 What a Strong Project Contribution Looks Like

A strong final project does more than demonstrate that an application can be built. It identifies a specific interaction failure or unmet user need, introduces an interaction mechanism that is meaningfully different from a standard chat, form, dashboard, or automation flow, and tests a claim about its effect. The claim can concern understanding, error detection, user control, accessibility, trust calibration, decision quality, safety, inclusion, workload, or task performance. Teams should compare their design against an appropriate baseline, such as a conventional interface, an answer-first assistant, an automatic workflow, or a static explanation. The final report should state what was learned, where the finding applies, and what remains uncertain.

4.2 Human-AI Interaction and Language Systems

1. **Uncertainty-Aware Multilingual Public-Service Assistant**
   Expected project items: Help Turkish-English speakers decide when an assistant's answer is sufficient and when they must verify or escalate a public-service question.
   Hardware component: Build a bilingual question flow that presents an answer, confidence cue, evidence links, and a clear human-service escalation route.
   Evaluation focus: Use a phone microphone for speech input and camera or QR scanning for document or service-code capture. A mobile or web interface in JavaScript/TypeScript or Flutter sends the input to the language service and makes confidence, evidence, and escalation states visible. Compare an answer-first assistant with the uncertainty-aware version for verification behaviour, error detection, task completion, and calibrated reliance. Evidence that explicit uncertainty and source control can improve appropriate reliance in a multilingual public-service interaction.

2. **Calibrated AI Feedback for Novice Programmers**
   Expected project items: Reduce unproductive dependence on generated debugging advice without making feedback slow or discouraging.
   Hardware component: Create an IDE-adjacent interface that changes the amount of hinting according to a learner's stated confidence and recent debugging actions.
   Evaluation focus: Collect keyboard or editor events in a browser or VS Code-style prototype; use a hardware button or vibration cue to let the learner request, defer, or silence support. JavaScript/TypeScript or Python maps the event stream to visible feedback states. Compare fixed full-answer feedback, fixed hints, and calibrated feedback for debugging success, time, explanation use, and perceived autonomy. A tested interaction policy for matching AI feedback depth to novice debugging needs.

3. **Cross-Lingual Explanation Consistency Inspector**
   Expected project items: Support users who receive materially different AI explanations in Turkish and English for the same question.
   Hardware component: Build a side-by-side inspector that segments claims, flags possible additions, omissions, or changed certainty, and lets the user request a plain-language comparison.
   Evaluation focus: Use microphone input and a physical language-toggle button or NFC card. A web interface calls translation and language-model services, aligns claims, and highlights differences rather than merely displaying two texts. Measure whether users identify important inconsistencies more accurately and explain their verification decision more clearly than with two unannotated responses. A usable method for making cross-lingual explanation inconsistency inspectable by non-expert users.

4. **Human-Controlled Source-Grounded Assistant**
   Expected project items: Give users meaningful control over which evidence an assistant may use

before it generates a response.
Hardware component: Build a retrieval-selection screen where users inspect, include, exclude, or rank sources and see how source choice changes the response.
Evaluation focus: Use QR-coded source cards or an NFC reader for tangible source selection. An ESP32 or phone NFC interface sends source identifiers to a Python or JavaScript retrieval service, which updates the browser interface and generation prompt. Compare default retrieval with user-selected retrieval for source verification, perceived agency, response quality, and time. Evidence about when source selection is worth the interaction cost in grounded language assistance.

5. **Agent Action Preview for High-Stakes Tasks**
Expected project items: Prevent harmful or surprising agent actions by making planned steps and permission consequences reviewable before execution.
Hardware component: Build an agent preview interface showing the next action, affected data, reversibility, and a clear approve, revise, pause, or cancel path.
Evaluation focus: Connect physical approve, pause, and cancel buttons to an ESP32 or Arduino. Button events reach a web interface through WebSocket, and the interface controls a simulated agent workflow written in Python or JavaScript. Compare auto-execution, text-only confirmation, and action preview for users' understanding, error prevention, and intervention timing. A tested interaction pattern for human control over agentic AI actions.

6. **Accessible Voice Prompting for Screen-Reader Users**
Expected project items: Design voice prompting and AI-response navigation for people who already rely on screen readers.
Hardware component: Build a voice-first workflow with short audio landmarks, structured response navigation, repeat and repair commands, and haptic confirmation.
Evaluation focus: Use the device microphone, screen-reader APIs, and phone vibration. Native Android, iOS, Flutter, or web accessibility APIs must expose every control and state change without requiring vision. Evaluate recoverability, completion time, navigation burden, and accessibility barriers against a conventional chat interface. Design evidence for voice and screen-reader interaction patterns that improve control of AI assistance.

7. **Interactive LLM Diversity Explorer**
Expected project items: Help users notice repeated ideas, missing alternatives, and overrepresented framings in generated outputs.
Hardware component: Build an interface that clusters multiple responses, exposes semantic overlap, and lets users ask for deliberately different alternatives.
Evaluation focus: Use a tangible dial to control diversity or viewpoint breadth and physical output cards to select clusters. An Arduino or ESP32 maps dial values and card IDs to a JavaScript visualisation and generation requests. Compare ordinary chat with the explorer for idea breadth, recognition of repetition, and quality of selected alternatives. A tangible-interaction approach for exposing output homogenisation in language-model ideation.

8. **Argument-Aware Discussion Companion**
Expected project items: Support students or community participants in forming replies that distinguish claims, evidence, and counterarguments.
Hardware component: Build a composition workspace where users arrange claims, evidence, limitations, and counterarguments before drafting a response.
Evaluation focus: Use RFID-tagged or QR-coded argument cards. A reader sends card IDs to a browser-based argument map, and an LLM may suggest missing evidence or counterarguments without writing the final reply automatically. Measure recognition of weak evidence, balance of replies, and perceived ownership compared with an ordinary AI drafting interface. Evidence that structured tangible argument work can reduce uncritical acceptance of generated arguments.

9. Speech Repair Assistant for Non-Native Speakers
Expected project items: Provide pronunciation and phrasing support while preserving speaker control and reducing embarrassment.
Hardware component: Build a practice flow offering immediate, delayed, or on-demand repair, playback, alternatives, and an explicit no-feedback mode.
Evaluation focus: Use microphone and headphones; a mobile or web application records speech, calls speech recognition or pronunciation analysis, and returns audio and visual feedback. A hardware button can switch feedback mode during speaking. Compare feedback timing conditions for fluency, repair uptake, comfort, and willingness to continue. A controlled comparison of feedback timing for speech-oriented AI support.

10. Mobile Provenance Verification Lens
Expected project items: Help users check claims encountered in physical or social contexts before accepting or sharing them.
Hardware component: Build a camera-based lens that captures a claim, identifies provenance cues, retrieves linked evidence, and prompts the user to record a verification decision.
Evaluation focus: Use the phone camera to capture printed or on-screen claims and QR markers. A mobile interface uses OCR and backend verification services, then presents provenance signals and a traceable decision path. Measure verification accuracy, time, source use, and sharing intention against a standard browser search flow. A situated mobile interaction for claim verification that links camera capture to accountable user decisions.

11. Cognitive-Load Adaptive Explanations
Expected project items: Give users shorter, clearer explanations when they are overloaded without hiding important uncertainty or alternatives.
Hardware component: Build an explanation interface that offers compact, expanded, and stepwise views, triggered by a voluntary workload check-in rather than covert inference.
Evaluation focus: Use a wearable or phone vibration prompt for the check-in. A mobile or web interface records the user's state and adapts explanation structure through JavaScript/TypeScript or Python while preserving an always-available full view. Compare static and user-adaptive explanations for workload, comprehension, error rate, and user control. Evidence on transparent, user-approved adaptation of AI explanation depth.

12. Bias Challenge Interface for Generative AI
Expected project items: Train users to recognise stereotypes, unsupported assumptions, and missing perspectives in generated content.
Hardware component: Build a short challenge flow in which users inspect outputs, identify a problem, request a revision, and compare the result with a more careful alternative.
Evaluation focus: Use printed challenge cards with QR codes. A phone web interface reads the code, loads the scenario, and logs the user's critique and revision choices; optional haptic feedback marks completed challenges. Measure bias recognition, revision quality, and retention against a brief information-only condition. An evaluated interactive method for helping users interrogate generative-AI outputs rather than passively consume them.

4.3 Embodied and Accessible Systems

13. Multimodal Campus Navigation for Mobility Diversity
Expected project items: Help campus visitors choose routes that fit their own mobility, sensory, and confidence needs.
Hardware component: Build a route planner that explains barriers, offers alternatives, and lets users set preferences such as step-free access, low-noise paths, or rest points.
Evaluation focus: Use GPS for route location, phone camera for optional obstacle capture, and

vibration for turn guidance. A mobile interface connects map APIs and a route-ranking service, making every route trade-off explicit. Compare conventional shortest-route navigation and the barrier-aware version for route understanding, confidence, and successful arrival. A personalised and explainable navigation interaction for mobility diversity.

14. Tangible AI Literacy Toolkit

Expected project items: Help students understand how data, uncertainty, bias, and human oversight shape an AI system.

Hardware component: Create a guided activity in which users assemble inputs, model choices, confidence states, and oversight decisions with physical objects linked to a digital explanation.

Evaluation focus: Use cards, sliders, tokens, or NFC tags connected through an Arduino, ESP32, or phone reader. A browser interface changes its simulated AI output and explanation when objects are moved or scanned. Compare the toolkit with an information sheet for conceptual understanding, recall, and ability to identify a limitation. An embodied learning interaction that makes otherwise hidden AI system properties manipulable.

15. Inclusive Smart Classroom Controller

Expected project items: Let students control local classroom conditions or participation preferences through multiple accessible modes.

Hardware component: Build a shared controller for light, sound, display, captioning, or participation requests with clear status and conflict handling.

Evaluation focus: Use an ESP32 or Arduino with buttons, LEDs, and a sensor. The controller communicates through Wi-Fi or serial USB with a web interface that logs requests, shows current state, and drives a safe simulated classroom device if necessary. Evaluate learnability, access across interaction modes, perceived inclusion, and recovery from conflicting requests. A concrete multimodal design for shared control in an inclusive learning environment.

16. Context-Sensitive Haptic Reminder System

Expected project items: Reduce intrusive reminders by giving users a meaningful choice to act, defer, or silence them.

Hardware component: Build a reminder interaction that explains why it appeared and lets users set boundaries for timing and interruption.

Evaluation focus: Use phone activity, calendar, and optional accelerometer signals with vibration output. A native or Flutter application implements the reminder logic and exposes sensor-based context and user choices in the interface. Compare fixed reminders and context-sensitive reminders for interruption cost, acceptance, missed tasks, and perceived control. Evidence about user-controlled context sensitivity in haptic reminder design.

17. Wearable-Supported Study Break Coach

Expected project items: Support sustainable study breaks without turning monitoring into surveillance.

Hardware component: Build a lightweight coach that detects prolonged inactivity or continuous work and asks the student to choose a break, postpone it, or turn the feature off.

Evaluation focus: Use smartwatch or phone accelerometer signals and haptic output. A mobile application processes local sensor events, displays a transparent activity summary, and avoids continuous cloud upload. Measure adherence, intrusiveness, usefulness, and privacy understanding against fixed timer reminders. A privacy-conscious wearable interaction for voluntary study-break support.

18. Accessible Form Completion With Haptic Guidance

Expected project items: Improve detection and recovery from form errors for users who cannot rely on visual colour or layout.

Hardware component: Build a form that communicates field status, error type, and next repair action through accessible text, sound, and distinct haptic patterns.
Evaluation focus: Use phone vibration and screen-reader services. A mobile or accessible web interface maps validation states to announced and haptic feedback, while a hardware button can replay the latest instruction. Compare visual-only feedback and multimodal feedback for error recovery, completion time, and confidence. Design evidence for haptic error guidance in accessible digital forms.

19. Physical Consent Tokens for Shared Devices
Expected project items: Make data permissions on shared devices easy to understand, set, and reverse for different people.
Hardware component: Build a shared-device flow in which each person presents a token, sees the active permissions, and can change or revoke them.
Evaluation focus: Use NFC or RFID cards and a reader connected to ESP32, Raspberry Pi, or phone NFC. The reader updates a local web interface and a simulated device-data service through REST or WebSocket. Evaluate permission clarity, mistaken-choice recovery, and understanding of data persistence against a standard settings menu. A tangible permission model for shared-device privacy decisions.

20. Embodied AI Check-In for Well-Being Support
Expected project items: Explore whether a physical, bounded check-in can give users more agency than a generic conversational interface.
Hardware component: Build a non-clinical check-in that asks a small number of user-controlled questions, offers coping or support resources, and clearly states its limits.
Evaluation focus: Use a microcontroller with light, button, and haptic output. A Python or JavaScript service handles only the selected interaction state and shows the same choices on a companion interface; no diagnosis or emergency advice is allowed. Compare screen-only and embodied versions for autonomy, comfort, understanding of limits, and willingness to use support resources. An ethical design comparison of embodied versus screen-only well-being check-ins.

21. Noise-Aware Quiet-Space Finder
Expected project items: Help students choose an appropriate study space while making the basis and uncertainty of recommendations visible.
Hardware component: Build a map or list that combines current local sound readings with user preferences such as quietness, distance, and accessibility.
Evaluation focus: Use the phone microphone to compute local sound-level features without storing raw audio. A mobile or web interface sends only summary values to a recommendation service and explains each ranking. Compare transparent recommendations and simple ranked lists for trust, choice quality, and privacy understanding. A transparent interaction for context-aware quiet-space recommendations.

22. Tangible Environmental Feedback
Expected project items: Make everyday energy, water, or waste choices easier to understand through immediate physical and digital feedback.
Hardware component: Build a small interactive installation in which users perform a choice and see its estimated consequence, uncertainty, and alternative action.
Evaluation focus: Use ESP32 or Arduino with a sensor, LED, dial, or button. The controller sends events to a JavaScript dashboard or Python service, which returns physical feedback and an on-screen explanation. Measure comprehension, recall, and intended behaviour against a dashboard-only condition. Evidence about when tangible feedback improves understanding of environmental trade-offs.

23. Camera-Assisted Accessibility Issue Reporter
Expected project items: Make it easier to report environmental barriers without collecting unnecessary personal or bystander data.
Hardware component: Build a mobile reporting flow that captures a barrier, guides privacy-preserving framing, collects location only when needed, and produces a structured report.
Evaluation focus: Use phone camera and location signal. The mobile interface performs local image checks or redaction where possible, then sends a structured report to a backend prototype and shows what information will be shared. Evaluate reporting accuracy, privacy comprehension, and usability against an ordinary photo-upload form. A privacy-aware interaction for citizen reporting of accessibility barriers.

24. Collaborative Tabletop Scheduling
Expected project items: Help small groups negotiate time and task trade-offs in a way that makes quieter members' constraints visible.
Hardware component: Build a tabletop or tangible planner where members place availability and priority tokens, inspect conflicts, and produce a shared schedule.
Evaluation focus: Use NFC tags, tokens, or a tabletop sensor connected to Raspberry Pi, Arduino, or ESP32. Token events update a shared browser interface through WebSocket and recompute scheduling options. Compare the system with ordinary chat or calendar scheduling for coordination quality, participation balance, and satisfaction. An embodied group-scheduling interaction that supports inclusive negotiation.

4.4 Trust, Safety, and Socially Situated AI

25. Misinformation Friction Before Sharing
Expected project items: Encourage deliberate verification before a user forwards a potentially unreliable claim.
Hardware component: Build a sharing flow that introduces a short, explainable pause and offers source inspection, a reflection prompt, and a choice to continue or stop.
Evaluation focus: Use phone vibration and notification interaction. A mobile or web prototype intercepts a simulated share action, records which verification controls were used, and provides a visible source trail. Compare a standard share flow and the friction flow for verification behaviour, sharing intention, and perceived manipulation. Evidence about which forms of interaction friction encourage verification without unacceptable annoyance.

26. Privacy-Visible Health Diary
Expected project items: Give people control over how an AI-generated summary is produced, retained, corrected, and shared.
Hardware component: Build a diary flow in which users record an entry, inspect the summary and extracted items, correct errors, set retention, and revoke sharing.
Evaluation focus: Use phone microphone or wearable input. A mobile interface transcribes or imports data, sends it to a summarisation service, and makes storage and sharing states explicit; the project must use non-sensitive test scenarios unless approval is obtained. Measure perceived control, summary-error correction, and understanding of retention against a conventional diary interface. A reversible and inspectable interaction model for AI-generated personal summaries.

27. Fairness-Aware Recommendation Kiosk
Expected project items: Help people understand and choose between recommendation utility, personal preference, and fairness-related trade-offs.
Hardware component: Build a kiosk that shows recommendations, their rationale, a controllable preference or fairness setting, and the consequences of changing it.
Evaluation focus: Use a tablet plus physical knobs, sliders, or buttons connected to Arduino or

ESP32. The hardware sends preference changes to a JavaScript visualisation or Python recommender simulation and updates both screen and physical feedback. Measure trust, fairness perception, decision quality, and ability to explain a trade-off against a standard ranked list. An interactive explanation design for user-facing recommendation trade-offs.

28. Safe Medication Information Clarifier
Expected project items: Help users understand non-emergency medication instructions without presenting the system as a diagnostic authority.
Hardware component: Build a plain-language clarification flow that explains label vocabulary, asks the user to check the original instruction, states uncertainty, and directs them to a pharmacist or clinician when needed.
Evaluation focus: Use phone microphone and text-to-speech. A mobile or web interface captures a question, accesses an approved scenario set, and presents spoken and readable explanations with an explicit escalation control. Measure comprehension, recognition of when to seek professional help, and unsafe overreliance against a plain text information page. A safety-oriented interaction pattern for health-information clarification with calibrated escalation.

29. Data Donation Decision Aid
Expected project items: Support genuinely informed decisions about contributing personal data to a project or service.
Hardware component: Build an interactive consent experience that explains purpose, data types, benefits, risks, retention, withdrawal, and a no-pressure refusal path.
Evaluation focus: Use tangible decision cards and QR interaction. A phone or web interface reads selected cards, reveals tailored explanations, and stores only test choices in a local prototype. Compare the aid with a standard consent form for comprehension, recall, and confidence that refusal remains possible. An evaluated design for informed and reversible data-donation decisions.

30. Culturally Responsive Crisis Information Interface
Expected project items: Make public information easier to understand across language and communication preferences during a simulated crisis scenario.
Hardware component: Build a scenario-based interface that adapts language, tone, format, and action sequence while preserving the same verified core information.
Evaluation focus: Use voice input and phone notifications. A mobile or web application switches between language and presentation modes, logs user choices, and checks that essential instructions remain consistent. Compare a single-format message and the responsive interface for comprehension, trust, and perceived respect. Evidence about culturally responsive presentation without changing the factual content of urgent information.

31. Human Override for Smart-Home Automation
Expected project items: Give residents a clear way to inspect, contest, and revise an automated home decision.
Hardware component: Build a simulation of an automation such as lighting, heating, or access control that shows the trigger, predicted consequence, and an override or rule-edit option.
Evaluation focus: Use an ESP32 or Arduino sensor and physical override button. Sensor events enter a web interface or Python service, which visualises the automation and sends the selected decision back to an LED or actuator simulation. Compare automatic control and explanation-plus-override for understanding, undesired-action prevention, and trust calibration. A tested human-override pattern for understandable smart-home automation.

32. Child-Friendly AI Transparency Toy
Expected project items: Help children understand that an AI-enabled object can listen, make a limited prediction, be uncertain, and require adult help.

Hardware component: Build a tangible toy interaction that visibly transitions between listening, processing, uncertain, and help-seeking states using age-appropriate language.
Evaluation focus: Use a microcontroller, LED, button, and optional microphone. The device communicates with a simple local or simulated classifier, while a companion screen can show the same state for adult facilitation. Measure children's understanding of system limits, uncertainty, and appropriate help-seeking through age-appropriate tasks. A child-centred tangible design for explaining AI limits without anthropomorphising authority.

33. Emotionally Safe Peer-Support Assistant
Expected project items: Explore a peer-support interaction that preserves user choice and makes its limits and escalation options unmistakable.
Hardware component: Build a bounded conversation or reflection flow with opt-out, user-selected prompts, resource signposting, and clearly visible emergency boundaries.
Evaluation focus: Use phone microphone, haptic cue, or physical check-in card. A mobile or web interface controls the conversation state and routes only to prewritten, reviewed scenario responses; it must not claim counselling capability. Compare the bounded interface with a generic chat flow for autonomy, perceived emotional safety, and help-seeking intention. Design evidence for safer interaction boundaries in peer-support AI systems.

34. Public-Service Queue Transparency Display
Expected project items: Reduce uncertainty and perceived unfairness in a simulated public-service queue.
Hardware component: Build a display that explains estimated waiting time, uncertainty, priority rules, and available alternatives instead of showing a single unexplained number.
Evaluation focus: Use a Raspberry Pi display with button or token input. A Python or JavaScript queue simulator updates the display through local APIs and lets users query what changed their estimate. Measure clarity, fairness perception, frustration, and decision quality against a simple queue-number display. An explainable public-display interaction for uncertain queue information.

35. LLM Agent Boundary Setter
Expected project items: Help users set and understand what an agent may do, what data it may access, and what actions remain prohibited.
Hardware component: Build a pre-task boundary-setting interface with task scope, data permissions, no-go zones, and a reviewable contract shown before the agent starts.
Evaluation focus: Use a physical mode switch, NFC card, or hardware button. The device sends selected boundaries to a browser interface through serial, Bluetooth, or WebSocket, and a simulated agent respects those constraints visibly. Compare default permissions and user-set boundaries for understanding, undesired-action prevention, and confidence in stopping the agent. A tangible and inspectable method for setting boundaries around agentic AI behaviour.

36. Explain-on-Demand for LLM Advice
Expected project items: Let users choose the amount and type of explanation they need rather than forcing a single long answer.
Hardware component: Build an advice interface that offers answer, assumptions, evidence, alternatives, counterarguments, and uncertainty as distinct on-demand layers.
Evaluation focus: Use smartphone vibration or a tangible explanation control such as a dial or button. Hardware events select layers in a JavaScript or Unity interface, which requests and displays the selected explanation without hiding the basic answer. Compare fixed full explanations and on-demand explanations for understanding, confidence calibration, time, and perceived overload. Evidence about explanation granularity as a user-controlled interaction resource.

5 Progress Presentation Requirements

5.1 Progress Presentation I (Week 7) - 7%

Every team member must present. Progress Presentation I should show that the project is well defined, feasible within the semester, and ready to move from planning into implementation.

The presentation should include:

- **Problem, target users, context, and project question - 25%**
  Clearly explain the problem the team is addressing, who will use the proposed system, in which context it will be used, and what specific question the project will examine.
- **User needs, design requirements, and rationale - 20%**
  Present the main user needs and the design requirements derived from interviews, observation, task analysis, personas, scenarios, expert input, or relevant sources. Explain the key design decisions made so far.
- **Comparable systems, baseline, or alternative approach - 15%**
  Identify existing systems or common interaction patterns relevant to the project. Define the baseline or comparison condition that will help the team assess whether the proposed design creates an improvement.
- **Early prototype and hardware process - 20%**
  Show an early prototype, storyboard, wireframe, interaction flow, or technical mock-up. Identify the proposed hardware set and explain the intended process: user action, device signal, communication method, software platform, and resulting interface response.
- **Evaluation plan, feasibility, and expected contribution - 20%**
  Explain who will take part in the evaluation, what tasks they will perform, what measures will be collected, and what comparison will be made. The team should also show that the scope, timeline, technical plan, and hardware access are realistic.

At the end of this presentation, the team should be able to explain exactly what it will build, who it is for, how the hardware will affect the interaction, and what the evaluation is expected to show.

5.2 Progress Presentation II (Week 10) - 8%

Every team member must present. Progress Presentation II should demonstrate that the project has moved beyond planning and that the main interaction is functioning.

The presentation should include:

- **Working prototype and task flow - 25%**
  Demonstrate the main user task from beginning to end. The team should show the prototype in operation and explain how the interface supports the intended user journey.
- **Hardware, software, and interface integration - 20%**
  Demonstrate the real hardware-mediated interaction. Explain what the user does physically, what event or signal is produced, how the signal reaches the software, and what changes in the interface or physical output as a result.
- **Evaluation carried out so far - 20%**
  Present the evaluation activities completed by Week 10. Explain the participants or evaluators, tasks, measures, and early observations. If the full evaluation has not yet been completed, explain what remains and why.

- **Design iteration supported by evidence - 20%**
  Show what changed after feedback, observation, user testing, or expert evaluation. The team should explain the issue identified, the design change made, and the reason for that change.
- **Preliminary findings, remaining risks, and final plan - 15%**
  Summarise the early findings, remaining technical or evaluation risks, and the concrete plan for completing the prototype, final evaluation, report, and presentation.

6 Final Prototype, Demo Day, and Final Presentation

6.1 Final Prototype Submission (Week 13)

Each team must submit a stable version of its prototype. The submission must include:

- a working prototype or accessible prototype link;
- a short setup and use guide;
- the relevant repository, application link, or installation file;
- a list of required hardware, software, accounts, and permissions;
- an architecture figure showing the hardware, software, and interface connection;
- a backup demonstration recording in case the live setup fails.

The prototype must support the intended interaction. A static interface, slide deck, video-only concept, or decorative hardware component does not meet the requirement.

6.2 Final Presentation and Live Demonstration (Week 14) - 10%

Every team member must participate in the final presentation. The presentation should explain the complete project process: the problem, target users, design requirements, prototype, hardware-mediated interaction, evaluation, findings, limitations, and next steps.

The live demonstration must show a realistic user task and the real hardware component in operation.

- **Project framing and overall story - 20%**
  Clearly communicate the problem, intended users, context of use, project question, and why the proposed interaction matters.
- **Prototype quality and live demonstration - 25%**
  Demonstrate a complete and understandable task flow. The hardware component must actively change, enable, or support the interaction.
- **Usability, accessibility, inclusion, privacy, safety, and ethics - 20%**
  Explain how the project addresses the needs of different users and how risks related to accessibility, inclusion, privacy, safety, and responsible use were considered.
- **Evaluation and findings - 20%**
  Present the evaluation method, participant or evaluator profile, key findings, and the design changes made in response to the evidence.
- **Communication, timing, and team participation - 15%**
  Present the project clearly, stay within the allocated time, prepare the demonstration carefully, and ensure equitable participation by all team members.

6.3 Interactive Demo Day Format

Demo Day will use an **HCI Experience Passport** model. Each team will operate a small interaction station during rotating rounds. Visitors will try the prototype, complete a short feedback activity, and receive a stamp or sticker in their passport.

Each team should collect at least **5-10** short public-feedback responses. These responses do not replace the planned user or expert evaluation. They provide an additional opportunity to observe real use, identify unexpected issues, and reflect on how the system performs outside the team's own presentation.

The final report should include a short reflection on the main feedback received during Demo Day and any final change or limitation identified through this process.

7 Final Report (Week 14) - 15%

The final report should document the complete HCI project process in a clear and structured form. It should include the following sections:

> Abstract; Problem Context; Target Users; Comparable Systems and Background; Project Question; User Needs and Design Requirements; Design Rationale; Prototype and Hardware Process; Evaluation Method; Results; Discussion; Ethics, Accessibility, Inclusion, Privacy, and Safety; Limitations; Next Steps; Individual Contributions; and References.

The *Prototype and Hardware Process* section must explain:

- the exact hardware set used;
- what the user physically does;
- what signal, event, or data is produced;
- how this reaches the software;
- which programming language, platform, or library is used;
- what changes in the interface or physical output;
- the fallback mode used if the hardware is unavailable.

The report will be evaluated according to the following criteria:

- **Problem framing, target users, comparable systems, and project question - 15%**
  The report clearly defines the problem, user group, context, comparison point, and intended project claim.
- **Evaluation method and ethical practice - 20%**
  The evaluation plan is appropriate for the project, clearly documented, and carried out with attention to consent, privacy, safety, and responsible treatment of participants and data.
- **Design rationale, prototype, and hardware documentation - 20%**
  The report explains the design decisions, interaction flow, technical architecture, and meaningful role of the hardware component.

- **Evaluation results, analysis, and design iteration - 25%**
  The report presents evidence clearly, interprets the findings carefully, and shows how feedback or results informed at least one design improvement.
- **Expected contribution, limitations, reproducibility, and writing quality - 20%**
  The report explains what the project contributes, where the findings apply, what the limitations are, how the prototype can be reproduced, and how each member contributed.

8 Non-Negotiable Completion Requirements

A visually polished interface alone is not sufficient for successful completion of the final project. Every team must provide:

- a working interactive prototype that supports a realistic user task;
- a meaningful hardware-mediated interaction connected to the software and interface;
- a clear problem definition, target-user context, and focused project question;
- documented user needs, design requirements, and design rationale;
- completed user evaluation or structured expert evaluation;
- at least one design iteration supported by evaluation evidence;
- explicit attention to usability, accessibility, inclusion, privacy, safety, and ethics;
- the final prototype package, final presentation and live demonstration, and final report;
- clear evidence that each team member contributed meaningfully to the project.

Fabricated data, plagiarism, unapproved collection of sensitive data, absence of a testable prototype, decorative hardware, or inability to document meaningful individual participation may result in failure of the relevant assessment component or of the project as a whole.

9 Ethics, Data, and Responsible Project Practice

Groups must minimise risk. Do not collect sensitive personal data without explicit instructor approval. Do not claim medical, legal, financial, or psychological diagnosis. De-identify participant data, avoid unnecessary identifiers, store materials securely, and report only aggregated or anonymised results. Participants must understand that participation is voluntary and that they may stop at any time without penalty.

For AI-enabled projects, teams must state the tool used, its role in the system, foreseeable harms, known limitations, provenance of external data, and how users can understand, correct, override, or decline AI outputs.

9.1 Ethics Statement and Committee Process

If a project may require an ethics statement or Ethics Committee approval, the group must meet with the instructor in **Weeks 4-5**. The team should bring its project objective, participant group, recruitment plan, data to be collected, consent process, risks, and data-storage plan. After this meeting, an Ethics Committee application will be prepared and submitted where required. No data collection that requires approval may begin before the applicable process has been completed.